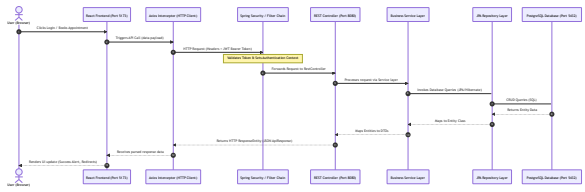
🏥 Hospital Appointment Management System

Complete Architectural Guide & REST API Reference Sheet

Welcome! This complete document details the architectural layout, connectivity parameters, and a thorough table-by-table CRUD API specification for your Spring Boot backend and React frontend application. Use the button above to print or save this guide as a PDF for offline study.

🗺️ System Architecture Overview

This project uses a decoupled **Client-Server Architecture**. The frontend and backend run as independent processes communicating over standard HTTP protocols exchanging JSON payloads.



🛠️ Technology Stack & Dependencies

- **Backend (Spring Boot 3.x / Java 21)**: Exposes RESTful endpoints, uses Spring Security with JWT filters for role authorization, and Spring Data JPA (Hibernate) to map object models to SQL.
- **Frontend (React.js / Vite / TailwindCSS)**: Component-based client with React Router for routing, Context API for authentication session states, and Axios for HTTP integration.
- **Database (PostgreSQL)**: Storage engine modeling entities such as Users, Doctors, Patients, Departments, and Appointments.

🌉 The Connection Bridge

React and Spring Boot communicate through two primary mechanisms:

A. Cross-Origin Resource Sharing (CORS)

Because they run on separate ports, the browser blocks standard HTTP calls unless specifically allowed. Spring Boot permits this inside the security config filter chain:

```
@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration config = new CorsConfiguration();
    config.setAllowedOrigins(List.of("http://localhost:5173")); // Trust React port
    config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "PATCH", "OPTIO
    config.setAllowedHeaders(List.of("*"));
    config.setAllowCredentials(true);
    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", config);
    return source;
}
```

B. Frontend Axios Settings

In the React application's `.env` file, the backend API path is declared: `VITE_API_BASE_URL=http://localhost:8080/api`. Axios instances consume this base URL and automatically inject JWT tokens into headers if they exist in localStorage.

📊 Complete Table-by-Table CRUD API Specification

This section documents every endpoint in the system, organized by their respective database tables. Every endpoint specifies the HTTP method, URL path, DTO payload, and necessary authentication role rules.

1. 👥 Users Table (System Access Accounts)

Stores system usernames, emails, roles, and hashed passwords.

> ***Table Name:*** `Users` | ***Spring Controller:*** `UserController.java` | ***React Service:*** `UsersServices.js`

GET /api/users — ADMIN ONLY

React Function: `getAllUsers()`

Spring Method: `getAllUsers()`

Response: `ApiResponse<List<UserResponseDTO>>`

GET /api/users/{id} — ADMIN ONLY

React Function: `getUserById(id)`

Spring Method: `getUserById(id)`

Response: `ApiResponse<UserResponseDTO>`

POST /api/users — PUBLIC / ADMIN

React Function: `createUser(data)`

Payload DTO: `UserRequestDTO`

Response: `ApiResponse<UserResponseDTO>`

PUT /api/users/{id} — ADMIN ONLY

React Function: `updateUser(id, data)`

Payload DTO: `UserRequestDTO`

Response: `ApiResponse<UserResponseDTO>`

`DELETE` **`/api/users/{id}`** `ADMIN ONLY`

React Function: `deleteUser(id)`

Spring Method: `deleteUser(id)`

Response: `ApiResponse<String>`

2. 🏥 Departments Table (Clinical Units)

Houses medical branch names and descriptions.

> ***Table Name:*** *`departments`* | ***Spring Controller:*** *`DepartmentController.java`* | ***React Service:*** *`DepartmentsServices.js`*

`GET` **`/api/departments`** `PUBLIC`

React Function: `getAllDepartments()`

Spring Method: `getAll()`

Response: `ApiResponse<List<DepartmentResponseDTO>>`

`GET` **`/api/departments/{id}`** `PUBLIC`

React Function: `getDepartmentById(id)`

Spring Method: `getById(id)`

Response: `ApiResponse<DepartmentResponseDTO>`

`POST` **`/api/departments`** `ADMIN ONLY`

React Function: `createDepartment(data)`

Payload DTO: `DepartmentRequestDTO`

Response: `ApiResponse<DepartmentResponseDTO>`

PUT **/api/departments/{id}** ADMIN ONLY

React Function: `updateDepartment(id, data)`

Payload DTO: `DepartmentRequestDTO`

Response: `ApiResponse<DepartmentResponseDTO>`

DELETE **/api/departments/{id}** ADMIN ONLY

React Function: `deleteDepartment(id)`

Spring Method: `delete(id)`

Response: `ApiResponse<String>`

3. 🥼 Doctors Table (Medical Staff Schedules)

Details doctor name, fee, specialization, active dates, and shifts.

> ***Table Name:*** *`doctors`* / ***Spring Controller:*** *`DoctorController.java`* / ***React Service:*** *`DoctorsServices.js`*

GET **/api/doctors** PUBLIC

React Function: `getAllDoctors()`

Spring Method: `getAllDoctors()`

Response: `ApiResponse<List<DoctorResponseDTO>>`

GET **/api/doctors/{id}** PUBLIC

React Function: `getDoctorById(id)`

Spring Method: `getDoctorById(id)`

Response: `ApiResponse<DoctorResponseDTO>`

`POST` **`/api/doctors`** ADMIN ONLY

React Function: `createDoctor(data)`

Payload DTO: `DoctorRequestDTO`

Response: `ApiResponse<DoctorResponseDTO>`

`PUT` **`/api/doctors/{id}`** ADMIN ONLY

React Function: `updateDoctor(id, data)`

Payload DTO: `DoctorRequestDTO`

Response: `ApiResponse<DoctorResponseDTO>`

`DELETE` **`/api/doctors/{id}`** ADMIN ONLY

React Function: `deleteDoctor(id)`

Spring Method: `deleteDoctor(id)`

Response: `ApiResponse<String>`

4. 🤯 Patients Table (Clinical Records)

Maintains patients demographic records and contact info.

> ***Table Name:*** *`patients`* | ***Spring Controller:*** *`PatientController.java`* | ***React Service:*** *`PatientsServices.js`*

`GET` **`/api/patients`** ADMIN ONLY

React Function: `getAllPatients()`

Spring Method: `getAllPatients()`

Response: `ApiResponse<List<PatientResponseDTO>>`

`GET` /api/patients/{id}

ADMIN ONLY

React Function: `getPatientById(id)`

Spring Method: `getPatientById(id)`

Response: `ApiResponse<PatientResponseDTO>`

`POST` /api/patients

ADMIN ONLY

React Function: `createPatient(data)`

Payload DTO: `PatientRequestDTO`

Response: `ApiResponse<PatientResponseDTO>`

`PUT` /api/patients/{id}

ADMIN ONLY

React Function: `updatePatient(id, data)`

Payload DTO: `PatientRequestDTO`

Response: `ApiResponse<PatientResponseDTO>`

`DELETE` /api/patients/{id}

ADMIN ONLY

React Function: `deletePatient(id)`

Spring Method: `deletePatient(id)`

Response: `ApiResponse<String>`

5. Appointments Table (Schedule Booking Details)

Links patients to doctors and logs fees, dates, and payment/booking states.

> ***Table Name:*** *`appointments`* | ***Spring Controller:*** *`AppointmentController.java`* | ***React Service:*** *`AppointmentsServices.js`*

GET **/api/appointments** ADMIN DOCTOR

React Function: `getAllAppointments()`

Spring Method: `getAll()`

Response: `ApiResponse<List<AppointmentResponseDTO>>`

GET **/api/appointments/{id}** ADMIN DOCTOR

React Function: `getAppointmentById(id)`

Spring Method: `getById(id)`

Response: `ApiResponse<AppointmentResponseDTO>`

POST **/api/appointments** ADMIN PATIENT

React Function: `createAppointment(data)`

Payload DTO: `AppointmentRequestDTO`

Response: `ApiResponse<AppointmentResponseDTO>`

PUT **/api/appointments/{id}** ADMIN DOCTOR

React Function: `updateAppointment(id, data)`

Payload DTO: `AppointmentRequestDTO`

Response: `ApiResponse<AppointmentResponseDTO>`

DELETE **/api/appointments/{id}** ADMIN DOCTOR

React Function: `deleteAppointment(id)`

Spring Method: `delete(id)`

Response: `ApiResponse<String>`

6. ✉ Contacts Table (Feedback & Inquiries)

Receives contact form submittals from guests and patients.

> ***Table Name:*** `contacts` | ***Spring Controller:*** `ContactController.java` | ***React Service:*** `Contact.js`

POST `/api/contacts` — PUBLIC

React Function: `createContact(data)`

Payload DTO: `ContactRequestDTO`

Response: `ApiResponse<ContactResponseDTO>`

GET `/api/contacts` — ADMIN ONLY

React Function: `getAllContacts()`

Spring Method: `getAll()`

Response: `ApiResponse<List<ContactResponseDTO>>`

GET `/api/contacts/{id}` — ADMIN ONLY

React Function: `getContactById(id)`

Spring Method: `getById(id)`

Response: `ApiResponse<ContactResponseDTO>`

DELETE **/api/contacts/{id}** ADMIN ONLY

React Function: `deleteContact(id)`

Spring Method: `delete(id)`

Response: `ApiResponse<String>`